

claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf\_095035a0-e9f\agent-acbf366bc061b1c0c.jsonl`
- SHA-256: `b461d7e96de78bd0a041a89c62610439d616080d129d7cd18ca3c66f94422251`
- Source modified: `2026-06-13T19:15:42+00:00`
- Imported at: `2026-07-08T16:00:00+00:00`
- project: `wf\_095035a0-e9f`
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`

Transcript

1. user (2026-06-13T19:13:53.216Z)

Adversarially VERIFY this fusion-P2 review finding by reading the actual code. Try to REFUTE it; only confirm real=true if it genuinely holds against the code. Default to real=false if uncertain or if it's a nitpick/false-positive.

FINDING: {"title":"frame_height has no <=0 fallback in detections_per_frame, and _person_features only back-fills width ? a 0/garbage container height silently zeroes every person feature","severity":"medium","file":"backend/pipeline/detectors/person_detector.py:282-283,300-301 and backend/pipeline/segmenters/fusion_hybrid.py:99-100","detail":"detections_per_frame guards fps (`if fps <= 0: fps = 30.0`, line 279-281) but does NOT guard frame_width/frame_height on the normal path (lines 282-283) ? unlike extract_action_windows_from_chunk which clamps frame_width to 1280 (line 162-164). It returns them raw. In _person_features the consumer does `fw = det['frame\_width'] or frame\_width` (falls back to the trajectory-probed width) but `fh = det['frame\_height'] or 0.0` ? NO real fallback for height. If a codec/container reports 0 (or the early-return path returns frame_height=0.0, person_detector.py:277), fh stays 0.0. Every fusion_features function that divides by height then short-circuits to 0: _foot_norm/_center_norm return (0,0) (fusion_features.py:54,61), person_in_court collapses, two_sided_motion/mean_player_motion/shuttle_player_colocation all return 0.0 (e.g. line 171). Result: the person cue silently produces all-zero features with no error ? and if min_players gating is on (_select_for_handoff, fusion_hybrid.py:124), every window with players_in_court=0 gets dropped from the AI handoff.", "fix":"Guard frame_width/frame_height against <=0 in detections_per_frame the way fps is guarded (e.g. clamp to a sensible default or to the probed values), AND/OR give fh a real fallback in _person_features (e.g. derive height from the probed width and aspect, or refuse the person cue and log when dims are unusable) instead of `or 0.0`.")}

Repo root = C:/Users/avidu/Projects/badminton-highlight-indexer. The change implements fusion P2 (MULTI_SIGNAL_FUSION_PLAN.md): a new default-OFF `FusionSegmenter` (backend/pipeline/segmenters/fusion_hybrid.py) that subclasses TrajectoryHybridSegmenter, persists shuttle net_crossings (Gate A) + optional person-cue features into candidate stats, and optionally gates the AI handoff. Pure feature math is in backend/pipeline/detectors/fusion_features.py. NEW files: backend/pipeline/detectors/fusion_features.py, backend/pipeline/segmenters/fusion_hybrid.py. EDITED files (use `git --no-pager diff -- <path>` to see edits): backend/pipeline/segmenters/trajectory_hybrid.py, backend/pipeline/detectors/person_detector.py, backend/config/models.py, backend/pipeline/segmenters/__init__.py. READ the actual files. The work is uncommitted in the working tree on branch master. Quote file:line. Only report REAL issues; rank by severity (high/med/low).

2. assistant (2026-06-13T19:13:56.223Z)

I'll verify this finding by reading the actual code. Let me start by examining the key files.

3. user (2026-06-13T19:13:57.193Z)

```

250         # Filter out extremely short action windows
251         min_window_duration = self.config.get("indexing",
252         {}).get("min_action_window_duration", 2.0)
253         windows = [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
254         return windows
255
256     def detections_per_frame(self, video_path: str, start_frame: int, end_frame: int,
257                             frame_skip: int = 3) -> Dict[str, Any]:
258         """Per-frame person detections over ``[start_frame, end_frame]`` of the ORIGINAL
259         video.
260
261         Used by the fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2). Unlike
262         ``extract_action_windows_from_chunk`` (which detects ITS OWN windows and
263         discards the
264         per-frame boxes), this returns the raw per-frame detections so the fusion
265         features can
266         align them with the shuttle trajectory. It reads the source by **absolute frame
267         index**
268         (seek + sequential read), NOT a re-extracted clip, so the returned frame indices
269         match
270         the WASB trajectory's ``.frame`` exactly ? that alignment is what
271         ``fusion_features.shuttle_player_colocation`` / ``two_sided_motion`` rely on.
272
273         Returns ``{"frames": {frame_idx: [(track_id, bbox), ...]}, "fps", "frame_width",
274         "frame_height"}``. A fresh window-local ByteTrack tracker is used. ? cv2 frame-
275         seek
276         accuracy varies by codec (it lands near the requested keyframe then decodes
277         forward);
278         treat the indices as ±a few frames on real footage ? fine for window-level
279         features.
280
281         """
282         self.lazy_load_model()
283         cap = cv2.VideoCapture(video_path)
284         if not cap.isOpened():
285             logger.error(f"Could not open {video_path}")
286             return {"frames": {}, "fps": 30.0, "frame_width": 0.0, "frame_height": 0.0}
287
288         fps = cap.get(cv2.CAP_PROP_FPS)
289         if fps <= 0:
290             fps = 30.0
291         frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
292         frame_height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
293         score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
294         0.5)
295
296         step = max(frame_skip, 1)
297         tracker = self.make_tracker(fps / step)
298
299         start_frame = max(0, int(start_frame))
300         frames: Dict[int, List[Tuple[int, Tuple[float, float, float, float]]]] = {}
301         cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
302         frame_idx = start_frame
303         while frame_idx <= end_frame:
304             ret, frame = cap.read()
305             if not ret:
306                 break
307             if (frame_idx - start_frame) % step == 0:
308                 frames[frame_idx] = self.detect_and_track(frame, tracker, score_thresh)
309             frame_idx += 1
310         cap.release()
311         return {"frames": frames, "fps": fps,
312                 "frame_width": float(frame_width), "frame_height": float(frame_height)}
313
314

```

```

1      """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3      Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4      TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5      do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6      shuttle-seeded window via two overridable seams the engine exposes:
7
8      - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9      GPU-free, computed from the trajectory we already have); and, only when the person cue
10     is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11     (``fusion_features``), persisting them into ``candidate_segments.stats`` for the learned
12     fusion (P3). The person path lazily builds ONE `PersonDetector` ? so the default path
13     never imports torch / touches the GPU.
14     - ``_select_for_handoff`` ? optional served Gate A/B: when
15     ``indexing.fusion.min_crossings``
16     (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
17     are
18     dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
19     **Persisted candidates remain the full superset** regardless, so the offline
20     experiment
21     harness can re-score any variant.
22
23     Defaults reproduce the substrate exactly: net_crossings/person features are persisted
24     but
25     nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
26     seeds
27     and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
28     default
29     segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
30     through the
31     experiment harness (EXPERIMENT_HARNESS.md), never silently.
32     """
33     from typing import Any, Dict, List, Optional, Tuple
34
35     from backend.pipeline.segmenters.base import segmenter_registry
36     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
37     from backend.pipeline.detectors.base import DetectorRunner
38     from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
39     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
40     from backend.pipeline.detectors import rally_gate
41     from backend.pipeline.detectors import fusion_features as ff
42
43     class FusionSegmenter(TrajectoryHybridSegmenter):
44         """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
45         B)."""
46
47         family = "fusion"
48         display_label = "Fusion"
49
50         def __init__(self, db: Any, config: Any):
51             super().__init__(db, config)
52             # Built lazily only if the person cue is enabled (keeps torch/GPU off the
53             default path).
54             self._person: Optional[Any] = None
55
56         @property
57         def name(self) -> str:
58             return "fusion_hybrid"
59
60         @property
61         def description(self) -> str:
62             return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
63             rallies, "
64
65             "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,

```

[illegible]

```

109     def _select_for_handoff(self, windows: List[Dict[str, Any]],
110                             window_stats: List[Dict[str, Any]],
111                             idx_cfg: Dict[str, Any]) -> List[int]:
112         """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
113         (0) keep every window ? identical to the substrate. Persisted candidates are
114         untouched (full superset)."""
115         fcfg = idx_cfg.get("fusion", {})
116         min_crossings = int(fcfg.get("min_crossings", 0))
117         min_players = int(fcfg.get("min_players", 0))
118         person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
119
120         keep: List[int] = []
121         for i, st in enumerate(window_stats):
122             if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
123                 continue
124             if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
125                 continue
126             keep.append(i)
127         return keep
128
129
130     segmenter_registry.register(FusionSegmenter)
131

```

5. user (2026-06-13T19:13:58.439Z)

```

1     """Pure per-window person-cue features for multi-signal rally fusion
(MULTI_SIGNAL_FUSION_PLAN §3.2/§4).
2
3     NO I/O, no torch, no cv2 ? mirrors `rally_gate.py`'s discipline so the whole feature
4     math is unit-testable in a GPU-free dev container. The detection (person boxes,
5     shuttle trajectory) is produced elsewhere (`PersonDetector`, the WASB runner); this
6     module only turns those signals into a handful of scale/translation-invariant
7     features (counts / ratios / zone-membership ? never raw pixel coordinates, which would
8     memorise this court/camera and fail to generalise off a small labelled set).
9
10    Conventions:
11    - A person box is `(x1, y1, x2, y2)` in pixels; `(track_id, box)` when motion
needs it.
12    - per_frame is `{frame_idx: [(track_id, box), ...]}` keyed by the ORIGINAL video
frame
13    index (so it aligns directly with the shuttle trajectory's frame ? no clip re-
timing).
14    - A shuttle point has frame, visible, x, y (pixels) ? the WASB
trajectory shape.
15    - court_polygon is normalised 0-1 [(x, y), ...] or None (None ? no mask,
every
16    detection counts ? the player-count-only mode).
17    - net is (axis 'x'|'y', position 0-1 normalised) or None (None ? two-sided =
0.0).
18
19    Every function degrades gracefully (returns 0.0 / empty) on missing inputs; outputs are
20    floats in [0, 1] except mean_player_motion (a non-negative normalised speed).
21    """
22    from __future__ import annotations
23
24    from typing import Dict, List, Optional, Sequence, Tuple
25
26    BBox = Tuple[float, float, float, float]
27    Net = Tuple[str, float]
28
29
30    def point_in_polygon(point: Tuple[float, float], polygon: Sequence[Tuple[float, float]])

```

```

-> bool:
31     """Ray-casting point-in-polygon. `point` and `polygon` share a coordinate space
32     (we use normalised 0-1). Empty/degenerate polygon (<3 vertices) ? False."""
33     if polygon is None or len(polygon) < 3:
34         return False
35     x, y = point
36     inside = False
37     n = len(polygon)
38     j = n - 1
39     for i in range(n):
40         xi, yi = polygon[i]
41         xj, yj = polygon[j]
42         # Does the horizontal ray at y cross edge (i, j)?
43         if (yi > y) != (yj > y):
44             x_cross = (xj - xi) * (y - yi) / (yj - yi) + xi if yj != yi else xi
45             if x < x_cross:
46                 inside = not inside
47         j = i
48     return inside
49
50
51     def _foot_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
52         """Normalised foot point (bottom-centre) of a person box ? where the player stands,
53         the right anchor for court membership. Returns (0,0) if frame dims are unusable."""
54         if frame_width <= 0 or frame_height <= 0:
55             return (0.0, 0.0)
56         x1, _y1, x2, y2 = box
57         return ((x1 + x2) / 2.0 / frame_width, y2 / frame_height)
58
59
60     def _center_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
61         if frame_width <= 0 or frame_height <= 0:
62             return (0.0, 0.0)
63         x1, y1, x2, y2 = box
64         return ((x1 + x2) / 2.0 / frame_width, (y1 + y2) / 2.0 / frame_height)
65
66
67     def person_in_court(box: BBox, frame_width: float, frame_height: float,
68                        court_polygon: Optional[Sequence[Tuple[float, float]]]) -> bool:
69         """Is this person on the court? Foot-point-in-polygon when a polygon is supplied;
70         True (count everyone) when it is None."""
71         if court_polygon is None:
72             return True
73         return point_in_polygon(_foot_norm(box, frame_width, frame_height), court_polygon)
74
75
76     def in_court_counts(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
77                        frame_height: float,
78                        court_polygon: Optional[Sequence[Tuple[float, float]]]) ->
List[int]:
79         """Per-frame count of in-court persons (one entry per sampled frame)."""
80         return [
81             sum(1 for _tid, box in dets if person_in_court(box, frame_width, frame_height,
court_polygon))
82             for _frame_idx, dets in sorted(per_frame.items())
83         ]
84
85
86     def _median(vals: Sequence[float]) -> float:
87         if not vals:
88             return 0.0
89         s = sorted(vals)
90         n = len(s)

```

```

91         return float(s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2]))
92
93
94     def players_in_court(counts: Sequence[int]) -> float:
95         """Median per-frame in-court count (?2 singles / 4 doubles; 0-1 = dead time)."""
96         return _median(list(counts))
97
98
99     def in_court_ratio(counts: Sequence[int], min_players: int = 2) -> float:
100         """Fraction of sampled frames with >= `min_players` in court (track stability vs
flicker)."""
101         if not counts:
102             return 0.0
103         return sum(1 for c in counts if c >= min_players) / len(counts)
104
105
106     def mean_player_motion(per_frame: Dict[int, List[Tuple[int, BBox]]],
107                           frame_width: float, frame_height: float) -> float:
108         """Mean per-track centre displacement between consecutive sampled frames, normalised
109         by the frame width (resolution-invariant). 0.0 if <2 frames or no shared tracks."""
110         if frame_width <= 0 or len(per_frame) < 2:
111             return 0.0
112         frames = sorted(per_frame.keys())
113         steps: List[float] = []
114         for a, b in zip(frames, frames[1:]):
115             prev = {tid: box for tid, box in per_frame[a]}
116             for tid, box in per_frame[b]:
117                 if tid in prev:
118                     cx0, cy0 = _center_norm(prev[tid], frame_width, frame_height)
119                     cx1, cy1 = _center_norm(box, frame_width, frame_height)
120                     steps.append(((cx1 - cx0) ** 2 + (cy1 - cy0) ** 2) ** 0.5)
121         if not steps:
122             return 0.0
123         return sum(steps) / len(steps)
124
125
126     def two_sided_motion(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
127                         frame_height: float, net: Optional[Net],
128                         court_polygon: Optional[Sequence[Tuple[float, float]]]) -> float:
129         """Fraction of sampled frames with >=1 in-court person on BOTH net halves.
130
131         A real rally is two-sided; a single player feeding/holding the shuttle is one-sided.
132         The net splits the play axis at `net=(axis, position)` in normalised coords. None ?
133         0.0.
134         """
135         if net is None or not per_frame:
136             return 0.0
137         axis, pos = net
138         both = 0
139         for _frame_idx, dets in per_frame.items():
140             near, far = False, False
141             for _tid, box in dets:
142                 if not person_in_court(box, frame_width, frame_height, court_polygon):
143                     continue
144                 fx, fy = _foot_norm(box, frame_width, frame_height)
145                 coord = fx if axis == "x" else fy
146                 if coord < pos:
147                     near = True
148                 else:
149                     far = True
150             if near and far:
151                 both += 1
152         return both / len(per_frame)
153

```

```

154     def _shuttle_in_box(sx: float, sy: float, box: BBox, frame_width: float, frame_height:
float,
155                        pad_frac: float) -> bool:
156         """Is the (pixel) shuttle point inside a person box, expanded by `pad_frac` of frame
157         width/height (the 'adjacent / in-hand' tolerance)?"""
158         x1, y1, x2, y2 = box
159         px = pad_frac * frame_width
160         py = pad_frac * frame_height
161         return (x1 - px) <= sx <= (x2 + px) and (y1 - py) <= sy <= (y2 + py)
162
163
164     def shuttle_player_colocation(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
165                                frame_width: float, frame_height: float,
166                                pad_frac: float = 0.03) -> float:
167         """Fraction of (visible shuttle frames that were also person-sampled) in which the
168         shuttle sits inside/adjacent to a person box. **held/in-hand ? high; in-flight ?
~0**
169         ? the direct held-shuttle discriminator. 0.0 if no aligned visible shuttle frames.
170         """
171         if not per_frame or frame_width <= 0 or frame_height <= 0:
172             return 0.0
173         aligned = 0
174         colocated = 0
175         for p in shuttle_points or []:
176             if not getattr(p, "visible", False):
177                 continue
178             dets = per_frame.get(int(p.frame))
179             if dets is None:          # shuttle frame wasn't person-sampled ? can't judge
180                 continue
181             aligned += 1
182             if any(_shuttle_in_box(p.x, p.y, box, frame_width, frame_height, pad_frac)
183                   for _tid, box in dets):
184                 colocated += 1
185         return colocated / aligned if aligned else 0.0
186
187
188     # The ~5 person features persisted into candidate `stats` (order = the learned-fusion
189     # feature columns appended after the shuttle features in distill_local.FEATURE_NAMES).
190     PERSON_FEATURE_NAMES = [
191         "players_in_court", "two_sided_motion", "mean_player_motion",
192         "in_court_ratio", "shuttle_player_colocation",
193     ]
194
195
196     def compute_person_features(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
197                                frame_width: float, frame_height: float,
198                                court_polygon: Optional[Sequence[Tuple[float, float]]] =
None,
199                                net: Optional[Net] = None,
200                                min_players: int = 2) -> Dict[str, float]:
201         """Aggregate the per-window person features from per-frame detections + the in-
window
202         shuttle trajectory. Pure; returns exactly `PERSON_FEATURE_NAMES`, all floats."""
203         counts = in_court_counts(per_frame, frame_width, frame_height, court_polygon)
204         return {
205             "players_in_court": players_in_court(counts),
206             "two_sided_motion": two_sided_motion(per_frame, frame_width, frame_height, net,
court_polygon),
207             "mean_player_motion": mean_player_motion(per_frame, frame_width, frame_height),
208             "in_court_ratio": in_court_ratio(counts, min_players),
209             "shuttle_player_colocation": shuttle_player_colocation(
210                 per_frame, shuttle_points, frame_width, frame_height),
211         }

```


6. assistant (2026-06-13T19:14:01.360Z)

Now let me look at the `extract\_action\_windows\_from\_chunk` clamp the finding references (line 162-164), and the early-return path.

7. user (2026-06-13T19:14:02.708Z)

```

140                                     frame_skip: int = 3,
141                                     chunk_index: Optional[int] = None) ->
List[Dict[str, Any]]:
142     """
143     Runs person detection + tracking on a video chunk and extracts high-motion
144     action windows. Returns a list of dicts in the full video's timeframe, each
with:
145         start, end (seconds), active_frame_count, peak_velocity, mean_velocity,
146         track_id_count, chunk_index.
147
148     Args:
149         frame_skip: Process every Nth frame (1 = every frame). Higher values
150                     dramatically reduce inference cost with minimal quality loss.
151         chunk_index: Index of the source chunk (recorded on each window for
traceability).
152     """
153     cap = cv2.VideoCapture(chunk_path)
154     if not cap.isOpened():
155         logger.error(f"Could not open {chunk_path}")
156         return []
157
158     fps = cap.get(cv2.CAP_PROP_FPS)
159     if fps <= 0:
160         fps = 30.0
161
162     frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
163     if frame_width <= 0:
164         frame_width = 1280.0 # Sensible fallback
165
166     # Effective sampling rate after frame-skipping
167     effective_fps = fps / max(frame_skip, 1)
168
169     # A fresh tracker per chunk ? track IDs only need to be consistent within a
170     # chunk (windows are computed per chunk, then merged across chunks by time).
171     tracker = self.make_tracker(effective_fps)
172     score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
0.5)
173
174     # Per active frame we keep (frame_idx, peak_velocity, set_of_active_track_ids)
175     # so windows can be enriched with motion stats for logging + fine-tuning.
176     active_frames = []
177     frame_idx = 0
178     track_history: Dict[int, Tuple[float, float, float, float]] = {}
179
180     while True:
181         ret, frame = cap.read()
182         if not ret:
183             break
184
185         # Skip frames for performance ? at 30fps, every 3rd frame still
186         # gives ~10 samples/second, plenty for human-scale velocity.
187         if frame_idx % max(frame_skip, 1) != 0:
188             frame_idx += 1
189             continue
190
191         # Detect persons + assign persistent track IDs.
```

```

192         detections = self.detect_and_track(frame, tracker, score_thresh)
193
194         frame_peak_velocity = 0.0
195         frame_active_ids = set()
196         current_ids = set()
197
198         for track_id, bbox in detections:
199             current_ids.add(track_id)

```

8. assistant (2026-06-13T19:14:08.511Z)

The finding's factual claims about the code are confirmed. Let me verify the key claim about consequences ? specifically whether the `fw` fallback to `frame\_width` is real but `fh` has no real fallback, and whether `mean\_player\_motion`/`shuttle\_player\_colocation`/`two\_sided\_motion` only guard on `frame\_width`.

Let me examine the consequence chain carefully. I want to verify:

1. Does `fw = det["frame\_width"]` or `frame\_width` provide a real fallback while `fh = det["frame\_height"]` or `0.0` does not?
2. When does `frame\_height` actually become 0 in practice?

Let me check the git diff to confirm these are the new edits, and look at how `frame\_width` (the trajectory-probed width) is obtained.

9. user (2026-06-13T19:14:09.716Z)

```

diff --git a/backend/pipeline/detectors/person_detector.py
b/backend/pipeline/detectors/person_detector.py
index 38ec2b3..5b931a4 100644
--- a/backend/pipeline/detectors/person_detector.py
+++ b/backend/pipeline/detectors/person_detector.py
@@ -252,3 +252,50 @@ class PersonDetector:
     windows = [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]

     return windows
+
+ def detections_per_frame(self, video_path: str, start_frame: int, end_frame: int,
+                          frame_skip: int = 3) -> Dict[str, Any]:
+     """Per-frame person detections over `[start_frame, end_frame)` of the ORIGINAL video.
+
+     Used by the fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2). Unlike
+     `extract_action_windows_from_chunk` (which detects ITS OWN windows and discards the
+     per-frame boxes), this returns the raw per-frame detections so the fusion features can
+     align them with the shuttle trajectory. It reads the source by **absolute frame index**
+     (seek + sequential read), NOT a re-extracted clip, so the returned frame indices match
+     the WASB trajectory's `.frame` exactly ? that alignment is what
+     `fusion_features.shuttle_player_colocation` / `two_sided_motion` rely on.
+
+     Returns `{"frames": {frame_idx: [(track_id, bbox), ...]}, "fps", "frame_width",
+     "frame_height"}`. A fresh window-local ByteTrack tracker is used. ? cv2 frame-seek
+     accuracy varies by codec (it lands near the requested keyframe then decodes forward);
+     treat the indices as ±a few frames on real footage ? fine for window-level features.
+     """
+     self.lazy_load_model()
+     cap = cv2.VideoCapture(video_path)
+     if not cap.isOpened():
+         logger.error(f"Could not open {video_path}")
+         return {"frames": {}, "fps": 30.0, "frame_width": 0.0, "frame_height": 0.0}
+
+     fps = cap.get(cv2.CAP_PROP_FPS)
+     if fps <= 0:
+         fps = 30.0
+
+     frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
+     frame_height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
+     score_thresh = self.config.get("indexing", {}).get("detector_score_threshold", 0.5)

```

```

+         step = max(frame_skip, 1)
+         tracker = self.make_tracker(fps / step)
+
+         start_frame = max(0, int(start_frame))
+         frames: Dict[int, List[Tuple[int, Tuple[float, float, float, float]]]] = {}
+         cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
+         frame_idx = start_frame
+         while frame_idx <= end_frame:
+             ret, frame = cap.read()
+             if not ret:
+                 break
+             if (frame_idx - start_frame) % step == 0:
+                 frames[frame_idx] = self.detect_and_track(frame, tracker, score_thresh)
+             frame_idx += 1
+         cap.release()
+         return {"frames": frames, "fps": fps,
+                 "frame_width": float(frame_width), "frame_height": float(frame_height)}

```

10. user (2026-06-13T19:14:10.555Z)

warning: in the working copy of 'backend/pipeline/segmenters/trajectory_hybrid.py', LF will be replaced by CRLF the next time Git touches it

warning: in the working copy of 'tests/test_trajectory_hybrid_equivalence.py', LF will be replaced by CRLF the next time Git touches it

```

backend/config/models.py | 34 ++++++
backend/pipeline/detectors/person_detector.py | 47 ++++++
backend/pipeline/segmenters/__init__.py | 2 +
backend/pipeline/segmenters/trajectory_hybrid.py | 50 ++++++-----
docs/README.md | 3 +-
tests/test_config.py | 30 ++++++
tests/test_trajectory_hybrid_equivalence.py | 22 ++++++
7 files changed, 177 insertions(+), 11 deletions(-)

```

11. assistant (2026-06-13T19:14:15.982Z)

The diff confirms the new `detections\_per\_frame` method exactly as described. Now let me verify a crucial aspect of the consequence chain ? whether `frame\_width` (the trajectory-probed width passed into `\_person\_features`) is itself reliably non-zero, since the finding claims `fw` has a real fallback but `fh` doesn't. Let me look at how `frame\_width` flows into `\_enrich\_candidate\_stats` from `trajectory\_hybrid.py`.

12. user (2026-06-13T19:14:16.802Z)

warning: in the working copy of 'backend/pipeline/segmenters/trajectory_hybrid.py', LF will be replaced by CRLF the next time Git touches it

```
diff --git a/backend/pipeline/segmenters/trajectory_hybrid.py
```

```
b/backend/pipeline/segmenters/trajectory_hybrid.py
```

```
index b2847c9..149282e 100644
```

```
--- a/backend/pipeline/segmenters/trajectory_hybrid.py
```

```
+++ b/backend/pipeline/segmenters/trajectory_hybrid.py
```

```
@@ -78,6 +78,31 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
```

```

    except (ValueError, TypeError):
        return max(3, int(duration / 0.8))

```

```

+ # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged) -----
+ def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
+                             frame_width: float, video_path: str,
+                             idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
+     """Stats dict persisted into ``candidate_segments.stats`` for one window. The BASE
+     returns exactly the 4 motion keys, so the trajectory twins persist byte-identical
+     rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
+     features). ``points`` are the whole-video trajectory points (TrajectoryPoint)."""
+     return {
+         "peak_velocity": cw["peak_velocity"],

```

```

+         "mean_velocity": cw["mean_velocity"],
+         "active_frame_count": cw["active_frame_count"],
+         "track_id_count": cw["track_id_count"],
+     }
+
+ def _select_for_handoff(self, windows: List[Dict[str, Any]],
+                         window_stats: List[Dict[str, Any]],
+                         idx_cfg: Dict[str, Any]) -> List[int]:
+     """Indices of the candidate windows that proceed to the AI handoff (and thus may
+     become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
+     ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
+     Persisted candidates are NOT affected ? they remain the full superset for offline
+     re-scoring."""
+     return list(range(len(windows)))
+
+ def process_video(self, video_id: str, video_path: str, # type: ignore[override]
+                  player_pool: Optional[List[str]] = None,
+                  max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
@@ -172,31 +197,36 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
+         f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
+         f"peak_v={cw['peak_velocity']:.3f}
mean_v={cw['mean_velocity']:.3f}")

+     # Per-window stats via the enrichment hook ? base = the 4 motion keys; the fusion
+     # segmenter adds net_crossings + person-cue features. Computed once, reused for both
+     # persistence and the handoff gate below.
+     window_stats = [
+         self._enrich_candidate_stats(cw, points, fps, frame_width, video_path, idx_cfg)
+         for cw in all_candidate_windows
+     ]
+
+     # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
+     candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
+     if store_candidates:
+         for i, cw in enumerate(all_candidate_windows):
-             stats = {
-                 "peak_velocity": cw["peak_velocity"],
-                 "mean_velocity": cw["mean_velocity"],
-                 "active_frame_count": cw["active_frame_count"],
-                 "track_id_count": cw["track_id_count"],
-             }
-             candidate_ids[i] = self.db.add_candidate_segment(
+                 video_id, detector=self.name,
+                 start_time=cw["start"], end_time=cw["end"],
+                 chunk_index=cw["chunk_index"], confidence=min(1.0, cw["peak_velocity"]),
+                 stats=stats, detection_params=detection_params,
+                 stats=window_stats[i], detection_params=detection_params,
+             )

+     if not all_candidate_windows:
+         return []

+     # --- Phase 3: AI provider handoff over padded candidate clips ---
+     # Which windows reach the handoff (base = all; fusion may apply Gate A/B). Persisted
+     # candidates above stay the full superset ? only the served play_segments are gated.
+     handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
idx_cfg)
+     handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
+     os.makedirs(handoff_dir, exist_ok=True)
+     logger.info("Phase 3: %s validation on %d candidate windows...",
-                 provider_name, len(all_candidate_windows))
+                 provider_name, len(handoff_indices))

+ def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:

```

```

        w_start = max(0, window["start"] - handoff_padding)
@@ -225,8 +255,8 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):

    ai_segments: List[dict] = []
    with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as executor:
        futures = [executor.submit(process_candidate_window, wi, cw)
-                   for wi, cw in enumerate(all_candidate_windows)]
+                   for wi, cw in enumerate(all_candidate_windows[wi])]
+                   for wi in handoff_indices]
        for future in futures:
            ai_segments.extend(future.result())
    try:

```

13. assistant (2026-06-13T19:14:20.277Z)

Now I need to find where `frame\_width` comes from in `process\_video` ? to determine whether the asymmetry between `fw` (which falls back to `frame\_width`) and `fh` (which falls back to 0) is real. Let me look at how `frame\_width` is probed in trajectory_hybrid's `process\_video`.

14. user (2026-06-13T19:14:22.019Z)

```

104         return list(range(len(windows)))
105
106     def process_video(self, video_id: str, video_path: str, # type: ignore[override]
107                      player_pool: Optional[List[str]] = None,
108                      max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
109         # override-ignore: the ABC declares the Result contract (Success|Failure)
110         # but every live segmenter still returns a bare list ? the documented
111         # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
112         label = self.display_label
113         # --- Sport profile + AI provider wiring (identical across segmenters) ---
114         sport_name = self.config.get("sport", "badminton")
115         try:
116             sport_profile = sport_registry.get(sport_name)
117         except KeyError as e:
118             logger.error(str(e))
119             return []
120
121         idx_cfg = self.config.get("indexing", {})
122         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
123 "gemini"))
124         model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
125 "gemini-2.5-flash"))
126
127         api_key_env_var = f"{provider_name.upper()}_API_KEY"
128         api_key = os.environ.get(api_key_env_var)
129         if not api_key:
130             logger.error(
131                 f"{api_key_env_var} environment variable is not set! "
132                 f"To run the {provider_name} handoff, set your API key. "
133                 f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
134             )
135             return []
136         try:
137             provider = provider_registry.get(provider_name, api_key=api_key,
138 model_name=model_name)
139         except KeyError as e:
140             logger.error(str(e))
141             return []
142
143         video_duration = get_video_duration(video_path)
144         if video_duration <= 0:
145             logger.error("Invalid video duration.")
146             return []
147
148         fps, frame_width = self._probe_fps_width(video_path)

```

```

145
146 # --- Runner config + windowing thresholds ---
147 runner, runner_params = self._build_runner(idx_cfg)
148
149 velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
150 merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
151 min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
152 handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
153 ai_max_workers = idx_cfg.get("ai_max_workers", 5)
154 log_candidates = idx_cfg.get("log_yolo_candidates", True)
155 store_candidates = idx_cfg.get("store_yolo_candidates", True)
156
157 detection_params = {
158     "detector": self.name,
159     "velocity_thresh": velocity_thresh,
160     "merge_gap": merge_gap,
161     "min_action_window_duration": min_window_duration,
162     **runner_params,
163 }
164
165 # --- Phase 1: env healthcheck (fail fast with a clear message) ---
166 ok, msg = runner.healthcheck()
167 if not ok:
168     logger.error("%s WSL environment not ready: %s", label, msg)
169     return []
170 logger.info("%s WSL env OK: %s", label, msg)
171
172 # --- Phase 2: trajectory -> candidate rally windows ---
173 # Trajectory detectors process the whole video in one pass (they carry
174 # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
175 t_start = time.time()
176 out_dir = os.path.join("output", self.family)
177 csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
178 if not csv_path:
179     logger.error("%s produced no trajectory; aborting.", label)
180     return []
181
182 points = runner.parse_trajectory_csv(csv_path)
183 logger.info("Parsed %d trajectory points (%d visible).",
184            len(points), sum(1 for p in points if p.visible))
185
186 all_candidate_windows = runner.trajectory_to_action_windows(
187     points, fps=fps, frame_width=frame_width, chunk_start=0.0,
188     velocity_thresh=velocity_thresh, merge_gap=merge_gap,
189     min_window_duration=min_window_duration, chunk_index=0,
190 )
191 logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
192            label, time.time() - t_start, len(all_candidate_windows))
193
194 if log_candidates:
195     for cw in all_candidate_windows:
196         logger.info(f"[{label}] rally]
197         {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
198             f"({cw['end'] - cw['start']:.1f}s) |
199 frames={cw['active_frame_count']} "
200             f"peak_v={cw['peak_velocity']:.3f}
201             mean_v={cw['mean_velocity']:.3f}")
202
203 # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
204 fusion
205 # segmenter adds net_crossings + person-cue features. Computed once, reused for
206 both
207 # persistence and the handoff gate below.
208 window_stats = [

```

15. assistant (2026-06-13T19:14:25.560Z)

Now let me check `\_probe\_fps\_width` to understand whether the trajectory-probed `frame\_width` itself can be 0, which matters for assessing how robust the `fw` fallback actually is.

16. user (2026-06-13T19:14:27.443Z)

```
58: def _probe_fps_width(video_path: str) -> tuple:
59:     """Return (fps, frame_width) for a video, with sensible fallbacks."""
60:     cap = cv2.VideoCapture(video_path)
61:     fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
62:     width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
63:     cap.release()
64:     return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
65:
66: @staticmethod
67: def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
68:     """Provider-reported exchange count, else a duration-based estimate.
69:
70:     One canonical implementation ? the twins had drifted (wasb relied on
71:     ``int(None)`` raising into the fallback; same result, by accident).
72:     """
73:     shot_count = segment.get("shuttle_exchange_count")
74:     if shot_count is None:
75:         return max(3, int(duration / 0.8))
76:     try:
77:         return int(shot_count)
78:     except (ValueError, TypeError):
79:         --
144:     fps, frame_width = self._probe_fps_width(video_path)
145:
146:     # --- Runner config + windowing thresholds ---
147:     runner, runner_params = self._build_runner(idx_cfg)
148:
149:     velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
150:     merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
151:     min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
152:     handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
153:     ai_max_workers = idx_cfg.get("ai_max_workers", 5)
154:     log_candidates = idx_cfg.get("log_yolo_candidates", True)
155:     store_candidates = idx_cfg.get("store_yolo_candidates", True)
156:
157:     detection_params = {
158:         "detector": self.name,
159:         "velocity_thresh": velocity_thresh,
160:         "merge_gap": merge_gap,
161:         "min_action_window_duration": min_window_duration,
162:         **runner_params,
163:     }
164:
```

17. assistant (2026-06-13T19:14:47.323Z)

This is highly confirmatory. Now I have the full picture. Let me verify the asymmetry is genuine and assess the consequence.

Key findings confirmed:

1. **`\_probe\_fps\_width` (line 58-64) explicitly clamps width: `width if width > 0 else 1280.0`** ? so the trajectory-probed `frame\_width` passed into `\_person\_features` is **guaranteed > 0** (always ? 1280.0). This means `fw = det["frame\_width"]` or `frame_width` (fusion_hybrid.py:99)` has a genuine, always-valid non-zero fallback.

2. **There is NO equivalent probe for height anywhere** ? `frame\_width` is the only dimension probed and threaded through `process\_video`. So `fh = det["frame_height"]` or `0.0``

(fusion_hybrid.py:100) has **only** `det["frame_height"]` as a source, and its fallback is literally `0.0`.

3. **detections_per_frame** (person_detector.py:282-283) returns raw `frame_width / frame_height` with no `<= 0` clamp on the normal path, unlike `extract_action_windows_from_chunk` which clamps width to 1280 (line 162-164). The early-return path (line 277) returns `frame_height: 0.0`.

Now let me verify the consequence chain ? that a 0 height genuinely zeroes the person features.

Confirmed from `fusion_features.py`:

- `_foot_norm` / `_center_norm` return `(0.0, 0.0)` if `frame_height <= 0` (lines 54, 61).
- `person_in_court` with a polygon calls `_foot_norm` ? always `(0,0)` ? `point_in_polygon` likely False ? `in_court_counts` all 0 ? `players_in_court` = 0.
- `two_sided_motion`: `_foot_norm` ? `(0,0)`, all on one side ? 0.0.
- `shuttle_player_colocation` guards `frame_height <= 0` ? returns 0.0 (line 171).
- `mean_player_motion` only guards `frame_width <= 0` (line 110), but `_center_norm` returns `(0,0)` so steps are all 0 ? 0.0.

So with `fh = 0.0`: `players_in_court=0`, `two_sided_motion=0`, `shuttle_player_colocation=0`, `mean_player_motion=0`. **All person features collapse to 0** silently.

Now the gating consequence: `_select_for_handoff` (fusion_hybrid.py:124) drops windows where `players_in_court < min_players` when `person_on` and `min_players > 0`. With `players_in_court=0`, **every** window is dropped from the AI handoff. This is real.

Let me verify one important nuance about the polygon-None case, since the finding's headline says "silently zeroes every person feature" ? let me check whether `players_in_court` is actually 0 even when `court_polygon` is None (the player-count-only mode).

18. user (2026-06-13T19:14:48.143Z)

```
67     def person_in_court(box: BBox, frame_width: float, frame_height: float,
68                           court_polygon: Optional[Sequence[Tuple[float, float]]]) -> bool:
69         """Is this person on the court? Foot-point-in-polygon when a polygon is supplied;
70         True (count everyone) when it is None."""
71         if court_polygon is None:
72             return True
73         return point_in_polygon(_foot_norm(box, frame_width, frame_height), court_polygon)
74
```

19. assistant (2026-06-13T19:15:04.860Z)

An important nuance for accuracy: when `court_polygon` is None, `person_in_court` returns `True` regardless of dims (short-circuits before `_foot_norm`). So `players_in_court` and `in_court_ratio` are **NOT** zeroed in the no-polygon mode. But `two_sided_motion` still calls `_foot_norm` to determine near/far ? collapses to one side ? 0.0. And `shuttle_player_colocation` still returns 0.0 (height guard). And `mean_player_motion` ? 0.0 (center collapses).

So the finding's claim "silently zeroes **every** person feature" is slightly overstated for the no-polygon mode ? `players_in_court` and `in_court_ratio` survive there. But:

- In the **polygon mode** (the headline scenario, and the mode where `min_players` court-gating is meaningful), `players_in_court` IS zeroed, so the gating drop-everything consequence is exactly real.
- In **all modes**, at least 3 of 5 features (`two_sided_motion`, `mean_player_motion`, `shuttle_player_colocation`) silently collapse to 0 with a 0 height.

This is a genuine, real bug ? the asymmetric fallback (`fw` has a real probed fallback clamped to ?1280, `fh` falls back to literal `0.0`) means a container/codecs reporting 0 height, or the early-return path, silently produces degenerate features with no error or log, and can drop every window from the handoff when court-gating is enabled. The fix suggestion (clamp dims in `detections_per_frame` like fps/width is clamped, and/or give `fh` a real fallback) is sound.

Severity assessment: The finding says "medium." Considerations that pull it down: this is a

default-OFF segmenter, the person cue is opt-in (`cue\_flags.person`), `min\_players` gating is also opt-in (default 0), and in practice most containers report a valid height via cv2. The trigger requires a codec/container that reports 0 height (uncommon but real, e.g. some streaming/variable formats, or the open-failure early-return). Considerations that keep it real: when it does trigger, it's a silent all-zero degradation with no log, and combined with `min\_players` gating it silently empties the handoff. The asymmetry vs. the width path is a genuine inconsistency. Medium is slightly high given the opt-in/opt-in gating and uncommon trigger; I'd call it low-to-medium, but it is a real defect, not a nitpick or false positive. I'll confirm real=true and note severity as low-medium.

Let me confirm there's no other height clamp I missed anywhere in the person cue path.

20. user (2026-06-13T19:15:05.982Z)

```
backend\pipeline\segmenters\fusion_hybrid.py:100:         fh = det["frame_height"] or 0.0
backend\pipeline\detectors\fusion_features.py:51:def _foot_norm(box: BBox, frame_width: float,
frame_height: float) -> Tuple[float, float]:
backend\pipeline\detectors\fusion_features.py:54:     if frame_width <= 0 or frame_height <= 0:
backend\pipeline\detectors\fusion_features.py:57:     return ((x1 + x2) / 2.0 / frame_width, y2 /
frame_height)
backend\pipeline\detectors\fusion_features.py:60:def _center_norm(box: BBox, frame_width: float,
frame_height: float) -> Tuple[float, float]:
backend\pipeline\detectors\fusion_features.py:61:     if frame_width <= 0 or frame_height <= 0:
backend\pipeline\detectors\fusion_features.py:64:     return ((x1 + x2) / 2.0 / frame_width, (y1
+ y2) / 2.0 / frame_height)
backend\pipeline\detectors\fusion_features.py:67:def person_in_court(box: BBox, frame_width:
float, frame_height: float,
backend\pipeline\detectors\fusion_features.py:73:     return point_in_polygon(_foot_norm(box,
frame_width, frame_height), court_polygon)
backend\pipeline\detectors\fusion_features.py:77:                                     frame_height: float,
backend\pipeline\detectors\fusion_features.py:81:                                     sum(1 for _tid, box in dets if
person_in_court(box, frame_width, frame_height, court_polygon))
backend\pipeline\detectors\fusion_features.py:107:                                     frame_width: float,
frame_height: float) -> float:
backend\pipeline\detectors\fusion_features.py:118:                                     cx0, cy0 =
_center_norm(prev[tid], frame_width, frame_height)
backend\pipeline\detectors\fusion_features.py:119:                                     cx1, cy1 = _center_norm(box,
frame_width, frame_height)
backend\pipeline\detectors\fusion_features.py:127:                                     frame_height: float, net:
Optional[Net],
backend\pipeline\detectors\fusion_features.py:141:                                     if not person_in_court(box,
frame_width, frame_height, court_polygon):
backend\pipeline\detectors\fusion_features.py:143:                                     fx, fy = _foot_norm(box,
frame_width, frame_height)
backend\pipeline\detectors\fusion_features.py:154:def _shuttle_in_box(sx: float, sy: float, box:
BBox, frame_width: float, frame_height: float,
backend\pipeline\detectors\fusion_features.py:160:     py = pad_frac * frame_height
backend\pipeline\detectors\fusion_features.py:165:                                     frame_width:
float, frame_height: float,
backend\pipeline\detectors\fusion_features.py:171:     if not per_frame or frame_width <= 0 or
frame_height <= 0:
backend\pipeline\detectors\fusion_features.py:182:     if any(_shuttle_in_box(p.x, p.y, box,
frame_width, frame_height, pad_frac)
backend\pipeline\detectors\fusion_features.py:197:                                     frame_width:
float, frame_height: float,
backend\pipeline\detectors\fusion_features.py:203:     counts = in_court_counts(per_frame,
frame_width, frame_height, court_polygon)
backend\pipeline\detectors\fusion_features.py:206:     "two_sided_motion":
two_sided_motion(per_frame, frame_width, frame_height, net, court_polygon),
backend\pipeline\detectors\fusion_features.py:207:     "mean_player_motion":
mean_player_motion(per_frame, frame_width, frame_height),
backend\pipeline\detectors\fusion_features.py:210:     per_frame, shuttle_points,
frame_width, frame_height),
backend\pipeline\detectors\person_detector.py:269:     "frame_height"]}``. A fresh window-
local ByteTrack tracker is used. ? cv2 frame-peek
```

```

backend\pipeline\detectors\person_detector.py:277:         return {"frames": {}, "fps": 30.0,
"frame_width": 0.0, "frame_height": 0.0}
backend\pipeline\detectors\person_detector.py:283:         frame_height =
cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
backend\pipeline\detectors\person_detector.py:301:         "frame_width":
float(frame_width), "frame_height": float(frame_height))

```

21. assistant (2026-06-13T19:15:11.077Z)

Confirmed. The only place `frame\_height` originates is `CAP\_PROP\_FRAME\_HEIGHT` in `detections\_per\_frame` (line 283, unclamped) and the early-return `0.0` (line 277). The only fallback at the consumer (fusion_hybrid.py:100) is `or 0.0`. There is no probe-based height fallback anywhere, in contrast to `frame\_width` which is clamped to 1280 both in `\_probe\_fps\_width` (line 64) and `extract\_action\_windows\_from\_chunk` (line 164). The asymmetry the finding describes is genuine.

I have fully verified the finding against the actual code.

22. user (2026-06-13T19:15:31.222Z)

Structured output provided successfully

23. assistant (2026-06-13T19:15:42.354Z)

Verified the finding against the actual code and confirmed it as real (severity low-medium).

Summary of what the code shows:

- `backend/pipeline/detectors/person\_detector.py:282-283` ? `detections\_per\_frame` returns `frame\_width`/`frame\_height` raw with no `<=0` clamp on the normal path, and the open-failure early-return at line 277 returns `frame\_height: 0.0`. fps IS guarded (lines 280-281), and `extract\_action\_windows\_from\_chunk` clamps width to 1280.0 (lines 162-164) ? so the missing height clamp is a genuine inconsistency.
- `backend/pipeline/segmenters/fusion\_hybrid.py:99-100` ? `fw = det["frame\_width"]` or `frame\_width` recovers because the probed `frame\_width` is always >0 (clamped to 1280 in `\_probe\_fps\_width`, trajectory_hybrid.py:64). But `fh = det["frame\_height"]` or `0.0` has no real fallback ? height is never probed anywhere else in the pipeline (grep confirms it only originates at person_detector.py:283/277).
- `backend/pipeline/detectors/fusion\_features.py` ? with fh=0, `\_foot\_norm`/`\_center\_norm` return (0,0) (lines 54,61), `shuttle\_player\_colocation` short-circuits to 0.0 (line 171), `two\_sided\_motion` and `mean\_player\_motion` collapse to 0.0. All silent.
- `backend/pipeline/segmenters/fusion\_hybrid.py:124` ? when the person cue and `min\_players>0` are both enabled, `players\_in\_court=0` drops every window from the AI handoff.

Two caveats that keep severity at low-medium rather than full medium: the segmenter is default-OFF and both the person cue and `min\_players` gating are opt-in (double gate to reach the worst outcome), and the "zeroes every feature" headline is exact only in polygon mode (in no-polygon mode `players\_in\_court`/`in\_court\_ratio` survive because `person\_in\_court` returns True before touching dims, fusion_features.py:71-72). Still a real defect ? a 0/garbage height silently degrades the person cue with no log ? not a nitpick.